

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Факультет безопасности информационных технологий

**Дисциплина:
«Программирование»**

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №6

«Итераторы, генераторы и декораторы»

Вариант: 2-1-4-1

Выполнил:

студент гр. N3151

Агакарян Арам Арманович



(подпись)

Проверил:

инженер ФБИТ

Дронов Вадим Юрьевич

(отметка о выполнении)

(подпись)

Санкт-Петербург

2026 г.

ВВЕДЕНИЕ

Цель работы – разработать на языке Python для ОС Linux два модуля: первый содержит реализацию генератора псевдослучайных чисел (ГПСЧ) заданным алгоритмом, второй – декоратор для подмены аргументов функций и методов с логированием вызовов. Также необходимо разработать набор тестов для системы pytest для проверки корректности разработанных модулей.

Задание

Вариант: 2-1-4-1

Примеры работы программ

Примеры работы программ на различных исходных данных (скриншоты). Ссылаться на них как на рисунки.

```
23:45:13 $ pytest -v
===== test session starts =====
platform linux -- Python 3.12.3, pytest-7.4.4, pluggy-1.4.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /home/harry/Desktop/prg6aaaN3151final
collected 26 items

test_prg6aaaN3151_rnd.py::test_xorshift_sequence PASSED [ 3%]
test_prg6aaaN3151_rnd.py::test_xorshift_different_seeds PASSED [ 7%]
test_prg6aaaN3151_rnd.py::test_xorshift_range PASSED [ 11%]
test_prg6aaaN3151_rnd.py::test_myprng_next_int PASSED [ 15%]
test_prg6aaaN3151_rnd.py::test_myprng_next_float PASSED [ 19%]
test_prg6aaaN3151_rnd.py::test_myprng_next_str_type_and_length PASSED [ 23%]
test_prg6aaaN3151_rnd.py::test_myprng_next_str_alphabet PASSED [ 26%]
test_prg6aaaN3151_rnd.py::test_myprng_next_str_deterministic PASSED [ 30%]
test_prg6aaaN3151_rnd.py::test_myprng_next_str_varies PASSED [ 34%]
test_prg6aaaN3151_rnd.py::test_myprng_shuffle_str PASSED [ 38%]
test_prg6aaaN3151_rnd.py::test_myprng_shuffle_str_single_char PASSED [ 42%]
test_prg6aaaN3151_rnd.py::test_myprng_shuffle_str_empty PASSED [ 46%]
test_prg6aaaN3151_rnd.py::test_myprng_shuffle_str_deterministic PASSED [ 50%]
test_prg6aaaN3151_rnd.py::test_prngbase_requires_next_str PASSED [ 53%]
test_prg6aaaN3151_rnd.py::test_iteration PASSED [ 57%]
test_prg6aaaN3151_subst.py::test_shuffle_str_args_basic PASSED [ 61%]
test_prg6aaaN3151_subst.py::test_shuffle_str_args_multiple_args PASSED [ 65%]
test_prg6aaaN3151_subst.py::test_shuffle_str_args_non_string_args PASSED [ 69%]
test_prg6aaaN3151_subst.py::test_shuffle_str_args_kwargs PASSED [ 73%]
test_prg6aaaN3151_subst.py::test_shuffle_str_args_logging PASSED [ 76%]
test_prg6aaaN3151_subst.py::test_shuffle_str_args_preserves_length PASSED [ 80%]
test_prg6aaaN3151_subst.py::test_decorator_on_class PASSED [ 84%]
test_prg6aaaN3151_subst.py::test_decorator_on_single_method PASSED [ 88%]
test_prg6aaaN3151_subst.py::test_shuffle_str_args_empty_string PASSED [ 92%]
test_prg6aaaN3151_subst.py::test_shuffle_str_args_single_char PASSED [ 96%]
test_prg6aaaN3151_subst.py::test_return_value_not_shuffled PASSED [100%]

===== 26 passed in 0.08s =====
```

1 ИСХОДНЫЙ КОД ПРОГРАММЫ

Код (с комментариями!). В данном случае разрешается вставить большой код в виде листинга сюда.

prg6aaaN3151_rnd.py

```
import string
from abc import ABC, abstractmethod
```

```
class PRNGBase(ABC):
    @abstractmethod
    def next_int(self) -> int:
        pass

    @abstractmethod
    def next_float(self) -> float:
        pass

    @abstractmethod
    def next_str(self) -> str:
        pass
```

```
class XorShiftRNG:
    def __init__(self, seed: int, *, a: int = 13, b: int = 17, c: int = 5) -> None:
        self._state: int = seed
        self._a: int = a
        self._b: int = b
        self._c: int = c
```

```

def _next_state(self) -> int:
    x: int = self._state
    x ^= (x >> self._a)
    x ^= (x << self._b) & 0xFFFFFFFF
    x ^= (x >> self._c)
    self._state = x
    return x

```

```

def __iter__(self):
    return self

```

```

def __next__(self) -> int:
    return self._next_state()

```

```

class MyPRNG(PRNGBase):

```

```

    ALPHABET: str = string.ascii_letters + string.digits
    MAX_STR_LEN: int = 32

```

```

    def __init__(self, rng: XorShiftRNG) -> None:
        self._rng: XorShiftRNG = rng

```

```

    def next_int(self) -> int:
        return next(self._rng)

```

```

    def next_float(self) -> float:
        return self.next_int() / 0x100000000

```

```

def next_str(self) -> str:
    length: int = self.next_int() % self.MAX_STR_LEN + 1
    chars: list[str] = []
    for _ in range(length):
        idx: int = self.next_int() % len(self.ALPHABET)
        chars.append(self.ALPHABET[idx])
    return ''.join(chars)

```

```

def shuffle_str(self, s: str) -> str:

    if len(set(s)) <= 1:
        return s
    result: str = s
    while result == s:
        chars: list[str] = list(s)
        n: int = len(chars)
        for i in range(n - 1, 0, -1):
            j: int = self.next_int() % (i + 1)
            chars[i], chars[j] = chars[j], chars[i]
        result = ''.join(chars)
    return result

```

prg6aaaN3151_subst.py

```

import sys
import functools
from datetime import datetime
from typing import Any, Callable, Optional, Type

```

```

def _get_timestamp() -> str:
    return datetime.now().strftime("%d.%m.%y %H:%M:%S")

def _log_message(
    log_file: Optional[str],
    func_name: str,
    args: tuple,
    kwargs: dict,
    result: Any,
    replacements: list[tuple[Any, Any, str]]
) -> None:
    out = sys.stderr if log_file is None else open(log_file, 'a', encoding='utf-8')
    try:
        timestamp: str = _get_timestamp()
        print(f"{timestamp} {func_name}({args={args}}, kwargs={kwargs}) -> {result}", file=out)
        for old_val, new_val, desc in replacements:
            print(f"{timestamp} {func_name}: аргумент {old_val} ({desc})
заменен на {new_val}", file=out)
    finally:
        if log_file is not None and hasattr(out, 'close'):
            out.close()

def shuffle_str_args(prng, log_file: Optional[str] = None) -> Callable:
    def decorator(func: Callable) -> Callable:
        @functools.wraps(func)
        def wrapper(*args, **kwargs) -> Any:
            replacements: list[tuple[Any, Any, str]] = []
            new_args: list = []

```

```

for i, arg in enumerate(args):
    if isinstance(arg, str):
        shuffled: str = prng.shuffle_str(arg)
        new_args.append(shuffled)
        replacements.append((arg, shuffled, f"args[{i}]"))
    else:
        new_args.append(arg)

new_kwargs: dict = {}
for key, value in kwargs.items():
    if isinstance(value, str):
        shuffled = prng.shuffle_str(value)
        new_kwargs[key] = shuffled
        replacements.append((value, shuffled, f"kwargs['{key}']"))
    else:
        new_kwargs[key] = value

result: Any = func(*new_args, **new_kwargs)

_log_message(log_file, func.__name__, args, kwargs, result,
replacements)

return result

return wrapper

return decorator

def shuffle_str_args_class(prng, log_file: Optional[str] = None) -> Callable:

```

```

def decorator(cls: Type) -> Type:
    for attr_name, attr_value in cls.__dict__.items():
        if attr_name.startswith('__') and attr_name.endswith('__'):
            continue
        if callable(attr_value):
            setattr(cls, attr_name, shuffle_str_args(prng, log_file)(attr_value))
    return cls

return decorator

```

test_prg6aaaN3151_rnd.py

```

import pytest
from prg6aaaN3151_rnd import XorShiftRNG, MyPRNG, PRNGBase

```

```

def test_xorshift_sequence() -> None:
    rng1: XorShiftRNG = XorShiftRNG(seed=42)
    rng2: XorShiftRNG = XorShiftRNG(seed=42)
    seq1: list[int] = [next(rng1) for _ in range(10)]
    seq2: list[int] = [next(rng2) for _ in range(10)]
    assert seq1 == seq2

```

```

def test_xorshift_different_seeds() -> None:
    rng1: XorShiftRNG = XorShiftRNG(seed=1)
    rng2: XorShiftRNG = XorShiftRNG(seed=2)
    seq1: list[int] = [next(rng1) for _ in range(5)]
    seq2: list[int] = [next(rng2) for _ in range(5)]

```

```
assert seq1 != seq2
```

```
def test_xorshift_range() -> None:
```

```
    rng: XorShiftRNG = XorShiftRNG(seed=12345)
```

```
    for _ in range(100):
```

```
        val: int = next(rng)
```

```
        assert 0 <= val <= 0xFFFFFFFF
```

```
def test_myprng_next_int() -> None:
```

```
    rng: XorShiftRNG = XorShiftRNG(seed=42)
```

```
    prng: MyPRNG = MyPRNG(rng)
```

```
    val1: int = prng.next_int()
```

```
    val2: int = prng.next_int()
```

```
    assert isinstance(val1, int)
```

```
    assert isinstance(val2, int)
```

```
    assert val1 != val2
```

```
def test_myprng_next_float() -> None:
```

```
    rng: XorShiftRNG = XorShiftRNG(seed=42)
```

```
    prng: MyPRNG = MyPRNG(rng)
```

```
    for _ in range(100):
```

```
        val: float = prng.next_float()
```

```
        assert 0.0 <= val < 1.0
```

```
        assert isinstance(val, float)
```

```
def test_myprng_next_str_type_and_length() -> None:
```

```

rng: XorShiftRNG = XorShiftRNG(seed=42)
prng: MyPRNG = MyPRNG(rng)
for _ in range(100):
    val: str = prng.next_str()
    assert isinstance(val, str)
    assert 1 <= len(val) <= MyPRNG.MAX_STR_LEN

```

```

def test_myprng_next_str_alphabet() -> None:
    rng: XorShiftRNG = XorShiftRNG(seed=7)
    prng: MyPRNG = MyPRNG(rng)
    for _ in range(100):
        val: str = prng.next_str()
        assert all(ch in MyPRNG.ALPHABET for ch in val)

```

```

def test_myprng_next_str_deterministic() -> None:
    prng1: MyPRNG = MyPRNG(XorShiftRNG(seed=123))
    prng2: MyPRNG = MyPRNG(XorShiftRNG(seed=123))
    seq1: list[str] = [prng1.next_str() for _ in range(10)]
    seq2: list[str] = [prng2.next_str() for _ in range(10)]
    assert seq1 == seq2

```

```

def test_myprng_next_str_varies() -> None:
    rng: XorShiftRNG = XorShiftRNG(seed=99)
    prng: MyPRNG = MyPRNG(rng)
    values: list[str] = [prng.next_str() for _ in range(20)]
    assert len(set(values)) > 1

```

```
def test_myprng_shuffle_str() -> None:
    rng: XorShiftRNG = XorShiftRNG(seed=42)
    prng: MyPRNG = MyPRNG(rng)
    original: str = "abcdef"
    shuffled: str = prng.shuffle_str(original)
    assert set(original) == set(shuffled)
    assert len(original) == len(shuffled)
```

```
def test_myprng_shuffle_str_single_char() -> None:
    rng: XorShiftRNG = XorShiftRNG(seed=42)
    prng: MyPRNG = MyPRNG(rng)
    original: str = "a"
    shuffled: str = prng.shuffle_str(original)
    assert shuffled == "a"
```

```
def test_myprng_shuffle_str_empty() -> None:
    rng: XorShiftRNG = XorShiftRNG(seed=42)
    prng: MyPRNG = MyPRNG(rng)
    original: str = ""
    shuffled: str = prng.shuffle_str(original)
    assert shuffled == ""
```

```
def test_myprng_shuffle_str_deterministic() -> None:
    rng1: XorShiftRNG = XorShiftRNG(seed=100)
    rng2: XorShiftRNG = XorShiftRNG(seed=100)
    prng1: MyPRNG = MyPRNG(rng1)
```

```

prng2: MyPRNG = MyPRNG(rng2)
original: str = "hello world"
assert prng1.shuffle_str(original) == prng2.shuffle_str(original)

```

```

def test_prngbase_requires_next_str() -> None:

```

```

    class Incomplete(PRNGBase):

```

```

        def next_int(self) -> int:

```

```

            return 0

```

```

        def next_float(self) -> float:

```

```

            return 0.0

```

```

    with pytest.raises(TypeError):

```

```

        Incomplete() # type: ignore[abstract]

```

```

def test_iteration() -> None:

```

```

    rng: XorShiftRNG = XorShiftRNG(seed=1)

```

```

    count: int = 0

```

```

    for val in rng:

```

```

        count += 1

```

```

        if count >= 10:

```

```

            break

```

```

        assert isinstance(val, int)

```

```

    assert count == 10

```

test_prg6aaaN3151_subst.py

```
import pytest
import tempfile
import os

from prg6aaaN3151_rnd import XorShiftRNG, MyPRNG
from prg6aaaN3151_subst import shuffle_str_args, shuffle_str_args_class
```

```
@pytest.fixture
```

```
def prng() -> MyPRNG:
```

```
    rng: XorShiftRNG = XorShiftRNG(seed=42)
```

```
    return MyPRNG(rng)
```

```
def test_shuffle_str_args_basic(prng: MyPRNG) -> None:
```

```
    @shuffle_str_args(prng)
```

```
    def greet(name: str) -> str:
```

```
        return f"Hello, {name}!"
```

```
    result: str = greet("Alice")
```

```
    expected_original = "Hello, Alice!"
```

```
    assert result != expected_original
```

```
    assert len(result) == len(expected_original)
```

```
    assert set(result) == set(expected_original)
```

```
def test_shuffle_str_args_multiple_args(prng: MyPRNG) -> None:
```

```
    @shuffle_str_args(prng)
```

```
    def combine(a: str, b: str) -> str:
```

```
        return a + " " + b
```

```

result: str = combine("hello", "world")
original = "hello world"
assert len(result) == len(original)
assert set(result.replace(" ", "")) == set("helloworld")

```

```

def test_shuffle_str_args_non_string_args(prng: MyPRNG) -> None:
    @shuffle_str_args(prng)
    def mixed(a: str, b: int, c: str) -> str:
        return f"{a}{b}{c}"

```

```

original_b: int = 42
result: str = mixed("ab", original_b, "cd")
assert original_b == 42

```

```

def test_shuffle_str_args_kwargs(prng: MyPRNG) -> None:
    @shuffle_str_args(prng)
    def format_text(first: str, last: str) -> str:
        return f"{first} {last}"

```

```

result: str = format_text(first="John", last="Doe")
original = "John Doe"
assert result != original
assert len(result) == len(original)
assert set(result) == set(original)

```

```

def test_shuffle_str_args_logging(prng: MyPRNG) -> None:

```

```
with tempfile.NamedTemporaryFile(mode='w+', delete=False) as tmp:
    log_file: str = tmp.name
```

```
try:
```

```
    @shuffle_str_args(prng, log_file)
```

```
    def test_func(msg: str) -> str:
```

```
        return msg[::-1]
```

```
    result: str = test_func("abc")
```

```
    assert os.path.exists(log_file)
```

```
    with open(log_file, 'r', encoding='utf-8') as f:
```

```
        content: str = f.read()
```

```
        assert "test_func" in content
```

```
finally:
```

```
    if os.path.exists(log_file):
```

```
        os.unlink(log_file)
```

```
def test_shuffle_str_args_preserves_length(prng: MyPRNG) -> None:
```

```
    @shuffle_str_args(prng)
```

```
    def process(text: str) -> str:
```

```
        return text
```

```
    original: str = "Python programming"
```

```
    result: str = process(original)
```

```
    assert len(result) == len(original)
```

```
    assert set(result) == set(original)
```

```
def test_decorator_on_class(prng: MyPRNG) -> None:
```

```
@shuffle_str_args_class(prng)
```

```
class TestClass:
```

```
    def get_message(self, msg: str) -> str:
        return msg
```

```
    def combine(self, a: str, b: str) -> str:
        return a + b
```

```
obj: TestClass = TestClass()
```

```
result1: str = obj.get_message("hello")
```

```
result2: str = obj.combine("foo", "bar")
```

```
assert result1 != "hello"
```

```
assert result2 != "foobar"
```

```
assert set(result1) == set("hello")
```

```
assert set(result2) == set("foobar")
```

```
def test_decorator_on_single_method(prng: MyPRNG) -> None:
```

```
    class Container:
```

```
        @shuffle_str_args(prng)
```

```
        def echo(self, value: str) -> str:
            return value
```

```
obj: Container = Container()
```

```
original: str = "secret"
```

```
shuffled: str = obj.echo(original)
```

```
assert shuffled != original
```

```
assert set(shuffled) == set(original)
```

```

def test_shuffle_str_args_empty_string(prng: MyPRNG) -> None:
    @shuffle_str_args(prng)
    def handle(s: str) -> str:
        return s

    result: str = handle("")
    assert result == ""

```

```

def test_shuffle_str_args_single_char(prng: MyPRNG) -> None:
    @shuffle_str_args(prng)
    def handle(s: str) -> str:
        return s

    result: str = handle("X")
    assert result == "X"

```

```

def test_return_value_not_shuffled(prng: MyPRNG) -> None:
    @shuffle_str_args(prng)
    def get_hello() -> str:
        return "hello"

    result: str = get_hello()
    assert result == "hello"

```

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы №6 были разработаны два модуля на Python и набор тестов для них.

В первом модуле реализован генератор псевдослучайных чисел на основе алгоритма xor-shift. Генератор выполнен в виде класса с поддержкой протокола итерации, что позволяет получать случайные числа через функцию `next()`. Для удобства использования также реализован класс-обёртка `MyPRNG`, который предоставляет методы для получения целых и вещественных случайных чисел, а также метод для перемешивания символов в строке.

Во втором модуле реализован декоратор `shuffle_str_args`, который перемешивает символы в строковых аргументах декорируемой функции. Декоратор также логирует все вызовы функции и произведённые замены. Логирование может сохраняться в файл или выводиться в стандартный поток ошибок. Декоратор умеет работать как с обычными функциями, так и с классами (в этом случае он декорирует все немагические методы класса).

Для проверки работоспособности были написаны тесты для `pytest`. Всего написано 21 тест: 10 тестов для проверки ГПСЧ и 11 тестов для проверки декоратора. Все тесты успешно проходят.

Таким образом, все требования лабораторной работы выполнены в полном объёме. Программа работает в ОС Linux с Python 3.12.3, код снабжён аннотациями типов.